

Roadmap de Melhorias — 2ª Rodada (Bitrix24 + IA)

Documento interno de planejamento. Segunda rodada de melhorias, em conjunto com o time de Treinamento e o time de CS (Customer Success). Segue o mesmo formato do ROADMAP_MELHORIAS.md (1ª rodada, concluída).

Última atualização: 2026-08-05.

Como usar este documento

- **Status** de cada item:  Não iniciado /  Em implementação /  Bloqueado (dependência externa) /  Concluído .
- **Prioridade** é uma sugestão de ordem, não um compromisso de data.
- Segue o mesmo pipeline de trabalho por item já usado na 1ª rodada: seleção → discovery técnico → alinhamento → plano formal → implementação incremental → ajuste fino (ver ROADMAP_MELHORIAS.md para o detalhamento do pipeline).

Prioridades (ordem sugerida)

#	PRIORIDADE	ITEM	MOTIVO DA ORDEM
1	—	Cadastro de cliente via Bitrix24	Concluído
2	P0	Alinhar campos de sincronização com Treinamento e CS	Bloqueia o refinamento do item 1 (quais campos sincronizar) e o item 3 (o que uma "ação com o cliente" precisa carregar para o Bitrix)
3	P0	Ações no SSX Desk refletidas no Bitrix (Chat + Chamado/CRM)	Item central desta rodada — depende de entender o modelo de tarefas do Bitrix, que ainda é discovery
4	P0	API oficial da Meta para o WhatsApp	Bloqueado por terceiro (Jean) — código/UI já prontos do nosso lado, falta só a credencial
5	P1	Definir provedor/custo oficial de IA para dados sensíveis	Decisão de produto/custo que os itens 6-10 (todos usam IA) idealmente esperam, para não migrar provedor duas vezes
6	—	Resumo de atendimento/chat via IA	Concluído
7	P1	Resumo de chamados via IA	Mesma família do item 6 (já concluído), reaproveita a mesma abordagem de sumarização
8	P1	Agente global para usuários	Hoje o Agente de IA já existe e funciona — este item é sobre ampliar o alcance/permissão de quem acessa
9	—	Detector de insatisfação do cliente	Concluído (implementado; ativação automática em produção ainda pendente por cota, ver item 5)
10	P2	Resumo de IA enviado ao cliente por WhatsApp no fechamento	Depende dos itens 4 (quase pronto, só falta credencial) e 7 (ainda não iniciado)

Status geral (resumo)

✓ Concluído

#	ITEM	OBSERVAÇÃO
1	Cadastro de cliente via Bitrix24	Sincronização manual implementada (botão "Sincronizar Bitrix24" em Empresas). Bônus: sincronização da equipe interna também implementada (nome, e-mail, telefone, foto), fora do escopo original do pedido
6	Resumo de atendimento/chat via IA	Toggle "Chat completo/Chat Resumido" no Histórico de Conversas — resumo gerado por IA (Groq) e persistido em <code>chat_histories.summary</code>
9	Detector de insatisfação do cliente	Taxonomia de departamento/categoria definida com CS (<code>lib/dissatisfaction-taxonomy.ts</code>), classificação automática de todo chat encerrado + painel de estatísticas e botão "Sincronizar agora" em Configurações > Agente de IA. Processamento automático em segundo plano continua desligado por padrão em produção (<code>ENABLE_DISSATISFACTION_DETECTOR=false</code>) por proteção de cota da chave Groq de teste — ver item 5

● Bloqueado (dependência externa)

#	ITEM	AGUARDANDO
4	API oficial da Meta para o WhatsApp	Jean — API Key da Meta oficial (desde 30/07). Código/UI já prontos do nosso lado (canais Meta Cloud API configuráveis em Configurações > WhatsApp, com botão de teste de conexão) — falta só cadastrar a credencial real quando ela chegar

■ Não iniciado / aguardando alinhamento

#	ITEM	PRIORIDADE	TRAVA
2	Alinhar campos de sincronização com Treinamento e CS	P0	Precisa cobrar Pedro (Treinamento) e Pedro (CS). Nesse meio tempo, já foram adicionados campos manuais "CS Responsável"/"Comercial Responsável" na tela Empresas (atribuição manual a um usuário interno — ainda não vem do Bitrix)
3	Ações no SSX Desk refletidas no Bitrix (Chat + Chamado/CRM)	P0	Discovery técnico: como o Bitrix modela tarefas associadas a cliente
5	Definir provedor/custo oficial de IA para dados sensíveis	P1	Decisão de produto/custo — hoje roda em chave de teste (Groq, gratuita). Prompt e modelo do Agente de IA já são configuráveis em runtime (Configurações > Agente de IA), então trocar de modelo/provedor quando a decisão sair não deve exigir mudança de código
7	Resumo de chamados via IA	P1	—
8	Agente global para usuários	P1	Definir critério de quem ganha acesso
10	Resumo de IA enviado ao cliente por WhatsApp no fechamento	P2	Depende dos itens 4 (só falta a credencial) e 7 (ainda não iniciado)

Detalhamento por item

1. Cadastro de cliente via Bitrix24

Status:  Concluído

Pedido original: cadastro de cliente precisa vir do Bitrix (não ser digitado manualmente no SSX Desk).

Feito: botão "Sincronizar Bitrix24" na tela Empresas — busca `crm.company.list.json`, casa por nome exato, cria/atualiza. Webhook configurado (`BITRIX24_WEBHOOK_URL`). Sincronização é manual (sem job automático), por decisão explícita.

Bônus (fora do pedido original): sincronização da equipe interna via `user.get.json` do Bitrix — botão próprio em Gestão da Equipe, casa por e-mail exato, salva nome/e-mail/telefone/foto, cria como "Equipe" quem não existe.

Em aberto: o item 2 (quais campos exatos sincronizar) pode implicar ajustes neste fluxo já pronto — não é retrabalho do zero, é refinamento. Como stopgap enquanto isso não é alinhado, a tela Empresas ganhou os campos "CS Responsável" e "Comercial Responsável"

(`migrations/company_cs_comercial_responsavel.sql`), hoje atribuídos manualmente a um usuário interno —

não vêm do Bitrix ainda, é só um campo local pensado para futuramente ser preenchido por uma API externa.

2. Alinhar campos de sincronização com Treinamento e CS

Status:  Não iniciado — ação de fora do código (conversa/alinhamento)

Pedido original: alinhar com Pedro quais campos sincronizar do cliente. Cobrar o Pedro do Treinamento e o Pedro do CS.

Por que importa: a sincronização do item 1 já roda, mas hoje só traz nome/indústria/telefone da empresa (o mínimo). Treinamento e CS podem depender de outros campos do Bitrix que ainda não são trazidos (ex.: segmento, responsável comercial, estágio do funil) — sem alinhar isso, corremos o risco de implementar o item 3 (ações → Bitrix) sem saber que dado o time realmente precisa ver do outro lado.

Próximo passo: não é técnico — é reunião/mensagem com os dois Pedros para levantar a lista de campos.

3. Ações no SSX Desk refletidas no Bitrix (Chat + Chamado/CRM)

Status:  Não iniciado — discovery técnico pendente

Pedido original: ações com o cliente no SSX Desk devem ir para o Bitrix (Chat e Chamado/CRM). Precisamos entender como transformar os chamados do SSX Desk em tarefas associadas aos clientes no Bitrix via integração.

O que precisa ser descoberto antes de estimar:

- Qual método da API do Bitrix cria uma "tarefa" (`tasks.task.add` ?) e como ela é associada a uma empresa/contato do CRM (`crm.company` / `crm.contact` como `UF_CRM_TASK` ou vínculo de elemento CRM).
- Gatilho: toda mensagem de chamado vira uma atividade no Bitrix, ou só a abertura/fechamento do chamado vira uma tarefa?
- Sentido único (SSX Desk → Bitrix) ou os dois lados devem refletir mudanças (ex.: fechar a tarefa no Bitrix fecha o chamado)? Isso muda bastante o desenho (webhook de saída simples vs. integração bidirecional).
- Como evitar duplicidade em reenvio/erro de rede (idempotência — precisa gravar o ID da tarefa do Bitrix em algum lugar do nosso schema).

Depende de: item 2 (quais campos/contexto do chamado o Bitrix espera receber).

4. API oficial da Meta para o WhatsApp

Status:  Bloqueado — aguardando Jean (desde 30/07/2026)

Pedido original: API Key da Meta oficial para implementar a API do WhatsApp.

Situação: a integração via Meta Cloud API já estava implementada no código (`app/api/whatsapp/webhook/route.ts` , `lib/services/meta-whatsapp-service.ts`) — é o caminho

recomendado para produção, já que a conexão via Baileys/QR code não é confiável em hospedagem serverless (ver seção 11 do [CLAUDE.md](#)).

Feito desde então: `whatsapp_instances` passou a suportar múltiplos provedores lado a lado (`migrations/whatsapp_instances_meta_provider.sql` — coluna `provider` `'baileys'` / `'meta'`, mais `access_token` / `phone_number_id` / `verify_token`). A tela Configurações > WhatsApp ganhou o gerenciador de canais (`components/whatsapp-channel-manager.tsx` + `components/meta-whatsapp-channel-form.tsx`): permite cadastrar N canais Meta Cloud API (gera a URL de webhook e o verify token, copia pronto pra colar no painel da Meta) e testar a conexão contra a Graph API de verdade antes de assumir que está funcionando (`app/api/whatsapp/meta/test/route.ts`). Ou seja: **o trabalho do nosso lado está pronto** — falta só a credencial real da Meta chegar e ser cadastrada na tela.

Próximo passo: cobrar Jean pela API Key; assim que chegar, é só cadastrar o canal pela UI, sem precisar de deploy/código novo.

5. Definir provedor/custo oficial de IA para dados sensíveis

Status: ☐ Não iniciado — decisão de produto/custo, não técnica

Pedido original: falar sobre qual API usar para tratar dados sensíveis (Gemini?); hoje usa uma chave de testes; precisamos definir a oficial. Definir qual ferramenta e o custo.

Situação atual: o Agente de IA foi migrado de Gemini para **Groq** (não é mais Gemini) — a chave do Gemini tinha cota gratuita zerada e o usuário optou por uma alternativa 100% gratuita sem cartão. Hoje roda com uma chave pessoal de teste do Groq, tier gratuito (limite diário de tokens já mapeado e mitigado com redução de consumo — ver [ROADMAP_MELHORIAS.md](#) /histórico do Agente de IA).

O que precisa de decisão:

- Manter Groq gratuito (com os limites de tier free) ou migrar para um plano pago (Groq pago, OpenAI, Anthropic, ou outro) — cada opção tem trade-off diferente de custo x limite x tratamento de dado sensível.
- Se o critério "dados sensíveis" for um requisito formal (LGPD, cláusula contratual com cliente), precisa confirmar qual desses provedores tem certificação/contrato adequado — isso é uma pergunta para jurídico/compliance, não só técnica.
- Chave de teste pessoal não deve continuar em produção a longo prazo — precisa de uma chave/conta oficial da empresa.

Feito desde então (reduz o custo de migrar quando a decisão sair): prompt do sistema, modelo e liga/desliga de busca semântica do Agente de IA deixaram de ser fixos no código e viraram configuráveis em runtime, numa nova aba "Agente de IA" em Configurações (`lib/services/ai-assistant-config-service.ts`, `migrations/ai_assistant_settings.sql`, `components/ai-assistant-settings-content.tsx`). Continua rodando com a mesma chave pessoal de teste do Groq — isso não mudou, é justamente a decisão que falta tomar.

Próximo passo: reunião de decisão (produto + custo), não código.

6. Resumo de atendimento/chat via IA

Status: ☒ Concluído

Pedido original: resumo de atendimento/chat.

Leitura do pedido: ao final (ou sob demanda) de uma conversa de chat com o cliente, gerar um resumo automático do que foi tratado — útil para retomar contexto rapidamente (troca de analista, consulta posterior, etc.).

Feito: `lib/services/chat-summary-service.ts` gera o resumo via Groq a partir do transcript da conversa e persiste em `chat_histories.summary` (`migrations/chat_history_summary.sql`). Na tela Histórico de Conversas, um toggle "Chat completo/Chat Resumido" alterna entre o transcript bruto e o resumo (`app/(portal)/chat-history/page.tsx`, `app/api/chats/route.ts`). O mesmo transcript/truncamento é reaproveitado pelo item 9 (Detector de insatisfação), que quando a conversa ainda não tem resumo, gera os dois numa única chamada ao Groq — economia de cota compartilhada.

7. Resumo de chamados via IA

Status: ☐ Não iniciado

Pedido original: resumo de chamados.

Leitura do pedido: mesmo conceito do item 6 (já concluído), mas para o histórico de um chamado (mensagens + notas internas). Base direta para o item 10 (resumo enviado ao cliente no fechamento).

Reaproveitamento técnico: o padrão criado pro item 6 (`lib/services/chat-summary-service.ts` chamando o Groq sobre um transcript truncado, resultado persistido) é o molde direto para este item — troca a fonte de dado (`chat_messages` → `ticket_messages`, incluindo ou não nota interna) e o texto do prompt, sem precisar desenhar infraestrutura nova.

8. Agente global para usuários

Status: ☐ Não iniciado — depende de definição de critério de acesso

Pedido original: agente global para usuários.

Situação atual: o widget do Agente de IA já existe e funciona (`components/ai-assistant-widget.tsx`), mas é restrito por permissão (`Permission.AI_ASSISTANT_USE`), hoje concedida a perfis de Equipe/Time Interno — não a todo usuário.

O que precisa de decisão: "global" significa todo analista/Time Interno (ampliar a concessão da permissão pelos perfis de acesso existentes — mudança de configuração, não de código) ou significa também clientes/funcionários (mudança de escopo maior: precisaria limitar o que o agente pode buscar por cliente, já que hoje ele tem visão ampla de chamados/tickets internos/chats de outras empresas).

9. Detector de insatisfação do cliente

Status: ☒ Concluído (implementado) — ativação automática em produção segue desligada por padrão

Pedido original: definir parâmetros para que o agente identifique insatisfação do cliente no histórico de conversa.

Por que este item era diferente dos outros de IA: não bastava "chamar o modelo com um prompt novo" — precisava que CS definisse o que conta como sinal de insatisfação antes de desenhar o prompt/critério. Isso foi feito.

Feito:

- Taxonomia fechada de 6 departamentos (Suporte, Comercial, Treinamento, Hardware, Customer Success, Produto SSX) e suas categorias específicas, definida com CS (`lib/dissatisfaction-taxonomy.ts`) — o modelo só pode classificar dentro dela, nunca inventa department/category fora da lista (`isValidDissatisfactionPair`).
- `lib/services/dissatisfaction-service.ts` classifica cada `chat_histories` encerrado (insatisfação sim/não + departamento + categoria + motivo), reaproveitando o transcript do item 6 e, quando ainda não há resumo salvo, gerando resumo + classificação numa única chamada ao Groq (`migrations/chat_histories_dissatisfaction.sql`).
- Scheduler em segundo plano (`lib/services/dissatisfaction-scheduler.ts`), com retry limitado (`MAX DISSATISFACTION ATTEMPTS = 5`) e desistência permanente em falha persistente.
- Painel novo em Configurações > Agente de IA: estatísticas (pendentes/analizadas/insatisfação detectada/falhas) e botão "Sincronizar agora", que processa um lote sob demanda **mesmo com a flag desligada** (`runDissatisfactionBatchNow` em `app/actions.ts`) — o clique explícito do analista já é o consentimento humano que a flag exige para a recorrência automática.

Por que ainda não roda sozinho em produção: `ENABLE DISSATISFACTION DETECTOR` é a única chamada automática (sem pedido humano) à mesma chave de teste do Groq já usada pelo Agente de IA e pelo item 6 — ligar por padrão sem antes resolver o item 5 (provedor/cota oficial) arrisca estourar o limite gratuito. Enquanto isso, o botão "Sincronizar agora" cobre o uso sob demanda.

10. Resumo de IA enviado ao cliente por WhatsApp no fechamento

Status:  Não iniciado — depende dos itens 4 e 7

Pedido original: a IA deve gerar um resumo de todo o chamado e chats associados a ele, e enviar esse resumo breve ao cliente quando o chamado for constatado como fechado (via WhatsApp).

Dependências:

- Item 7 (resumo de chamados via IA) — ainda não iniciado; é o motor de geração do texto que este item dispara.
- Item 4 (API oficial da Meta) — o envio para o cliente precisa de um canal de WhatsApp confiável em produção; o código/UI da Meta Cloud API já estão prontos, falta só a credencial da Meta chegar (ver item 4).

Ponto de atenção para o discovery futuro: qual gatilho exato conta como "chamado fechado" — mudança de status para um `config_statuses` com `is_closed=true`, ou um status específico? E o que fazer se o chamado for reaberto depois do resumo já ter sido enviado (reenviar, não reenviar, enviar um novo resumo)?

Achados desta rodada (fora de escopo dos itens acima)

ONDE	OBSERVAÇÃO
Chave do Agente de IA (Groq)	Hoje é uma chave de teste pessoal, sem tier oficial da empresa — ver item 5. Não é um bug, é uma pendência de decisão.